

Learning Search Space Partition for Black-box Optimization using Monte Carlo Tree Search

-- NeurIPS 2020

Hongming Zhang
2021.10.06

Introduction



- Idea: LA-MCTS solves high dimensional black-box optimization by combining **Monte Carlo Tree Search** (MCTS) with optimization methods like **Bayesian Optimization** (BO).

Motivation



- High dimensional black-box optimization suffers the **curse of dimensionality**. **Space partitioning** is a natural direction to address the issue.
- Using **MCTS (space partitioning)** combining with existing methods such as **BO (local modeling)** can learn the partition that is adaptive to the objective function $f(x)$ and balance exploration and exploitation.

Algorithm details



Object: $x^* = \operatorname{argmax}_{x \in X} f(x)$

$f(x)$:

Local smoothness such as Lipschitz continuity:

$$||f(x_1) - f(x_2)|| \leq C ||x_1 - x_2||$$

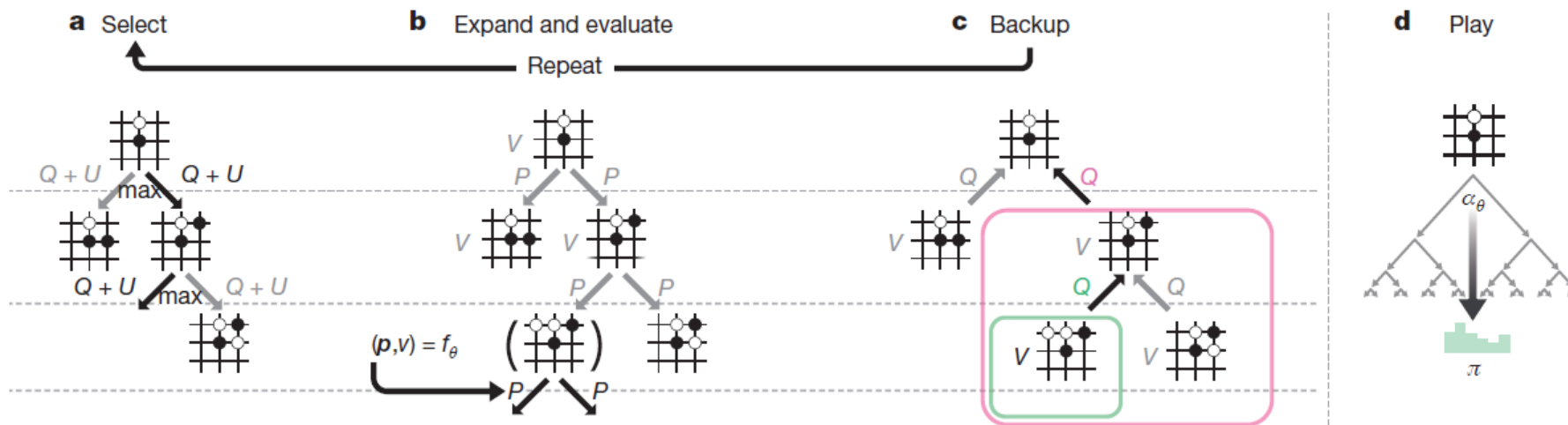
Consider the deterministic case.

Table 1: Definition of notations used through this paper.

$\mathbf{x}_i$	the i th sample	$f(\mathbf{x}_i)$	the evaluation of $\mathbf{x}_i$	D_t	collected $\{\mathbf{x}_i, f(\mathbf{x}_i)\}$ from iter $1 \rightarrow t$
Ω	the entire search space	Ω_j	the partition represented by node j	$D_t \cap \Omega_j$	samples classified in Ω_j
n_j	#visits at node j	v_j	the value of node j	ucb_j	the ucb score of node j

Algorithm details

- Dynamic tree construction via splitting
- Selecting via UCB
- Sampling via Bayesian Optimizations



Compare with MCTS in AlphaGo Zero

Algorithm details

- Dynamic tree construction via splitting

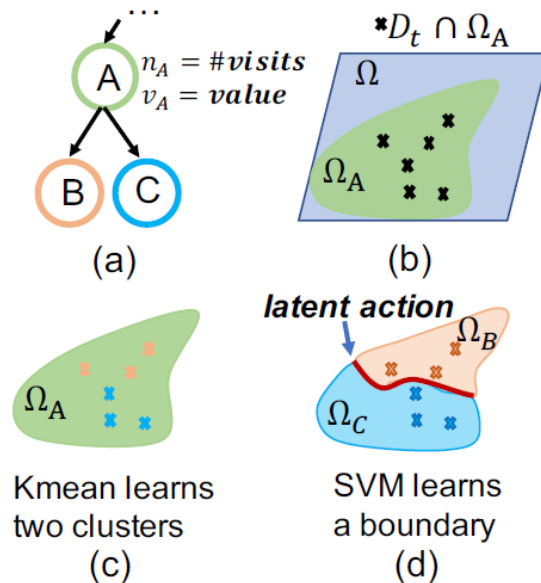


Figure 1: **the model of latent actions**: each tree nodes represents a region in the search space, and *latent action* splits the region into a high-performing and a low-performing region using $\mathbf{x}$ and $f(\mathbf{x})$.

$$n_A: \#samples \text{ in } D_t \cap \Omega_A$$
$$v_A = \frac{1}{n_i} \sum f(x_i), \forall x_i \in D_t \cap \Omega_A$$

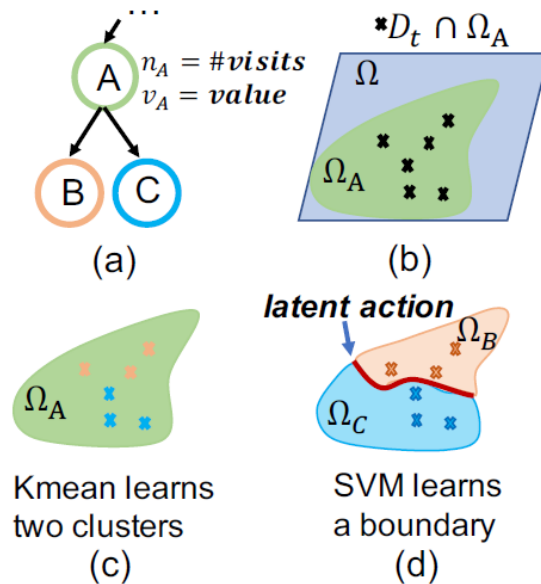
If the size of $D_t \cap \Omega_A$ exceeds the threshold θ at any leaves, we split the leaf with latent actions.

Use K-means to learn two clusters of different performance. Left child to represent a good region.

Learn a boundary with SVM

Algorithm details

- Selecting via UCB



n_A : #samples in $D_t \cap \Omega_A$

$$v_A = \frac{1}{n_i} \sum f(x_i), \forall x_i \in D_t \cap \Omega_A$$

Upper Confidence Bound (UCB)

$$ucb_B = v_B + 2C * \sqrt{\frac{2\log(n_A)}{n_B}}$$

Figure 1: **the model of latent actions**: each tree nodes represents a region in the search space, and *latent action* splits the region into a high-performing and a low-performing region using $\mathbf{x}$ and $f(\mathbf{x})$.

Algorithm details

- Sampling via Bayesian Optimizations

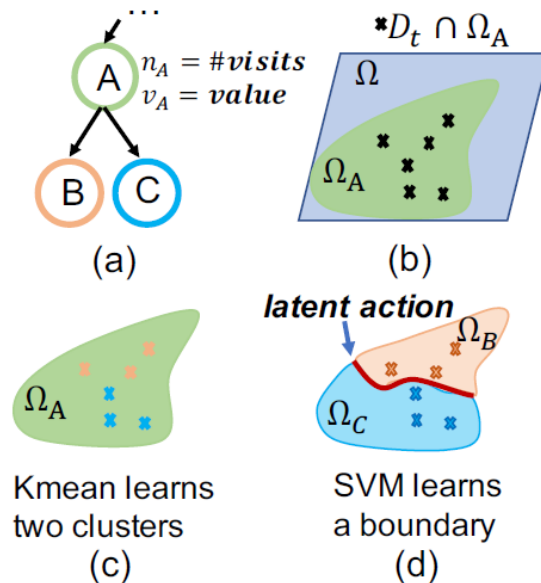


Figure 1: **the model of latent actions**: each tree nodes represents a region in the search space, and *latent action* splits the region into a high-performing and a low-performing region using $\mathbf{x}$ and $f(\mathbf{x})$.

Splitting constructs the tree and provides boundary for each node

selecting finds a path from the root to leaf node

Sampling here searches promising point in leaf node (local modeling)
See Bayesian Optimization in additional slides

Algorithm details

- Dynamic tree construction via splitting
- Select via UCB
- Sampling via Bayesian Optimizations

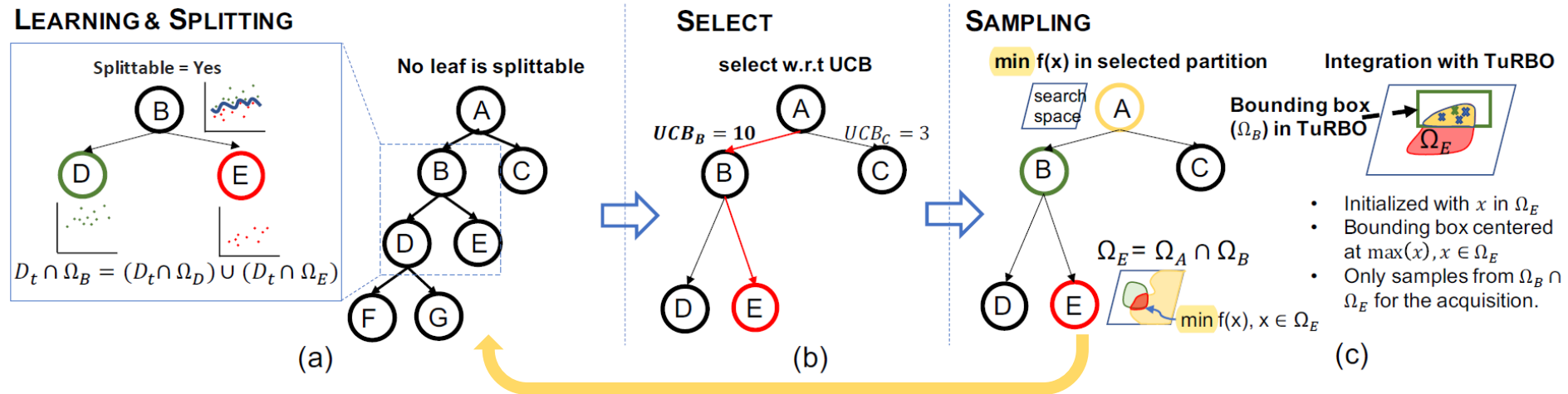


Figure 2: **the workflow of LA-MCTS**: In an iteration, LA-MCTS starts with building the tree via splitting, then it selects a region based on UCB. Finally, on the selected region, it samples by BO.

All the evaluations, i.e. x_i and $f(x_i)$ are returned to LA-MCTS to refine learned boundaries in the next iteration.

Experiment and analysis

- Mujoco locomotion tasks

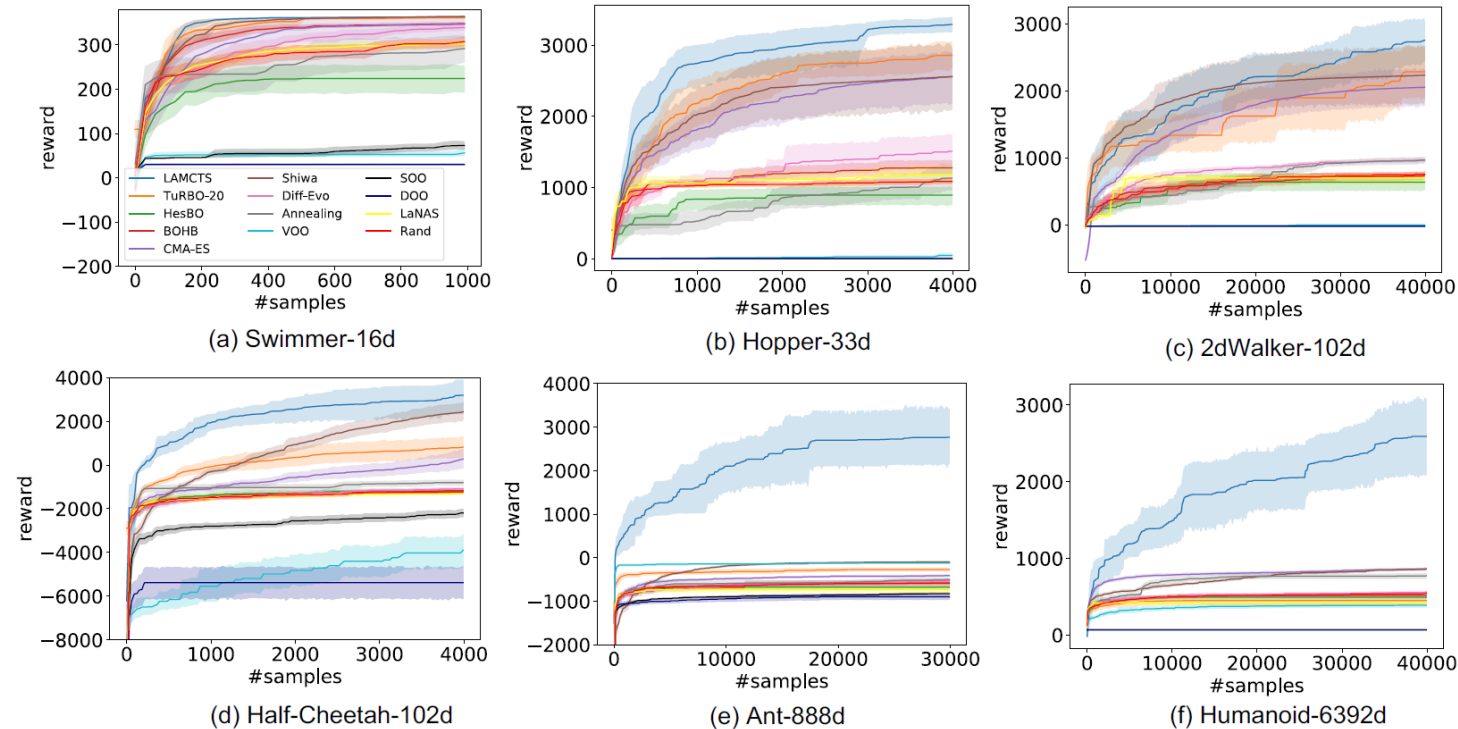


Figure 3: **Benchmark on MuJoCo locomotion tasks:** LA-MCTS consistently outperforms baselines on 6 tasks. With more dimensions, LA-MCTS shows stronger benefits (e.g. Ant and Humanoid). This is also observed in Fig. 4. Due to exploration, LA-MCTS experiences relatively high variance but achieves better solution after 30k samples, while other methods quickly move into local optima due to insufficient exploration.

Choose the linear policy $a = Ws$.

Each component of W is continuous and in the range of $[-1, 1]$.

Experiment and analysis

- Synthetic functions

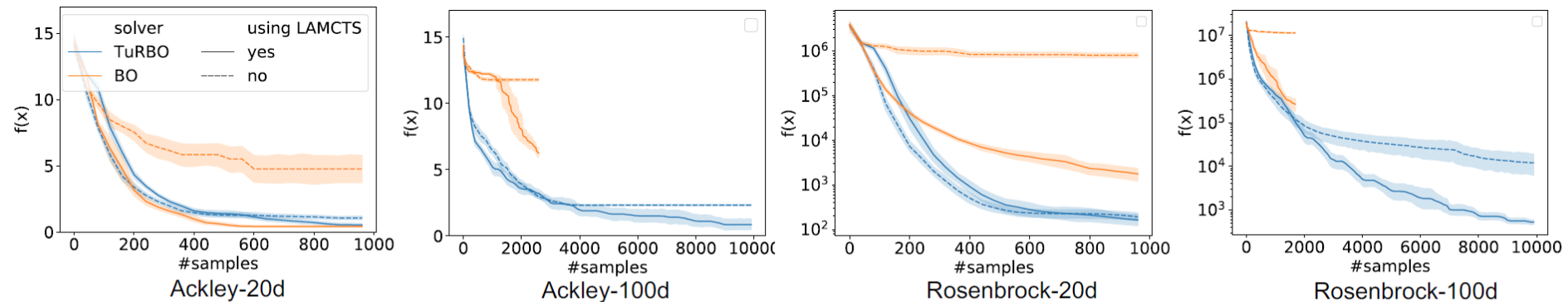


Figure 4: **LA-MCTS as an effective meta-algorithm.** LA-MCTS consistently improves the performance of TuRBO and BO, in particular in high-dimensional cases. We only plot part of the curve (collected from runs lasting for 3 day) for BO since it runs very slow in high-dimensional space.

Experiment and analysis

- 2d-Ackley

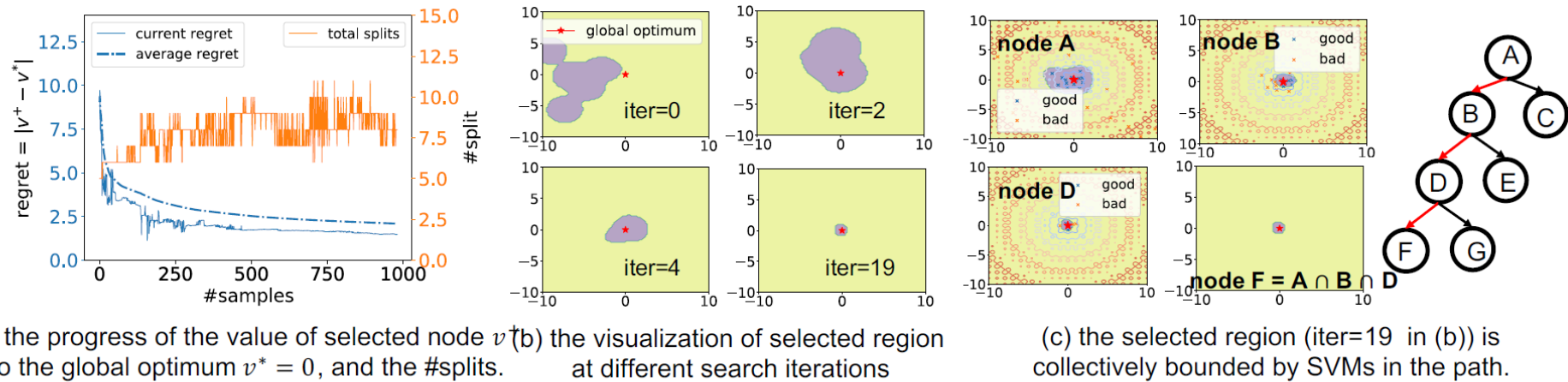


Figure 5: **Validation of LaMCTS:** (a) the value of selected node becomes closer to the global optimum as #splits increases. (b) the visualization of $\Omega_{selected}$ in the progress of search. (c) the visualization of $\Omega_{selected}$ that takes the intersection of nodes on the selected path.

Summary



LA-MCTS learns to partition the search space for Bayesian Optimization so that it can attend on a promising region to avoid over-exploring.

LA-MCTS is an effective meta-method to improve BO and significantly outperforms various existing black-box solvers

References



- Bayesian optimization and Gaussian process

Brochu E, Cora V M, De Freitas N. A tutorial on Bayesian optimization of expensive cost functions, with application to active user modeling and hierarchical reinforcement learning[J]. arXiv preprint arXiv:1012.2599, 2010.

https://www.bilibili.com/video/BV1L4411U76h?from=search&seid=13125415357471818631&spm_id_from=333.337.0.0

- MCTS and NAS

Wang L, Zhao Y, Jinnai Y, et al. Neural architecture search using deep neural networks and monte carlo tree search[C]. Proceedings of the AAAI Conference on Artificial Intelligence. 2020, 34(06): 9983-9991.

Wang L, Xie S, Li T, et al. Sample-efficient neural architecture search by learning actions for monte carlo tree search[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2021.

Zhao Y, Wang L, Tian Y, et al. Few-shot neural architecture search[C]. International Conference on Machine Learning. PMLR, 2021: 12707-12718.

- Github

<https://github.com/facebookresearch/LaMCTS>

Additional Slides

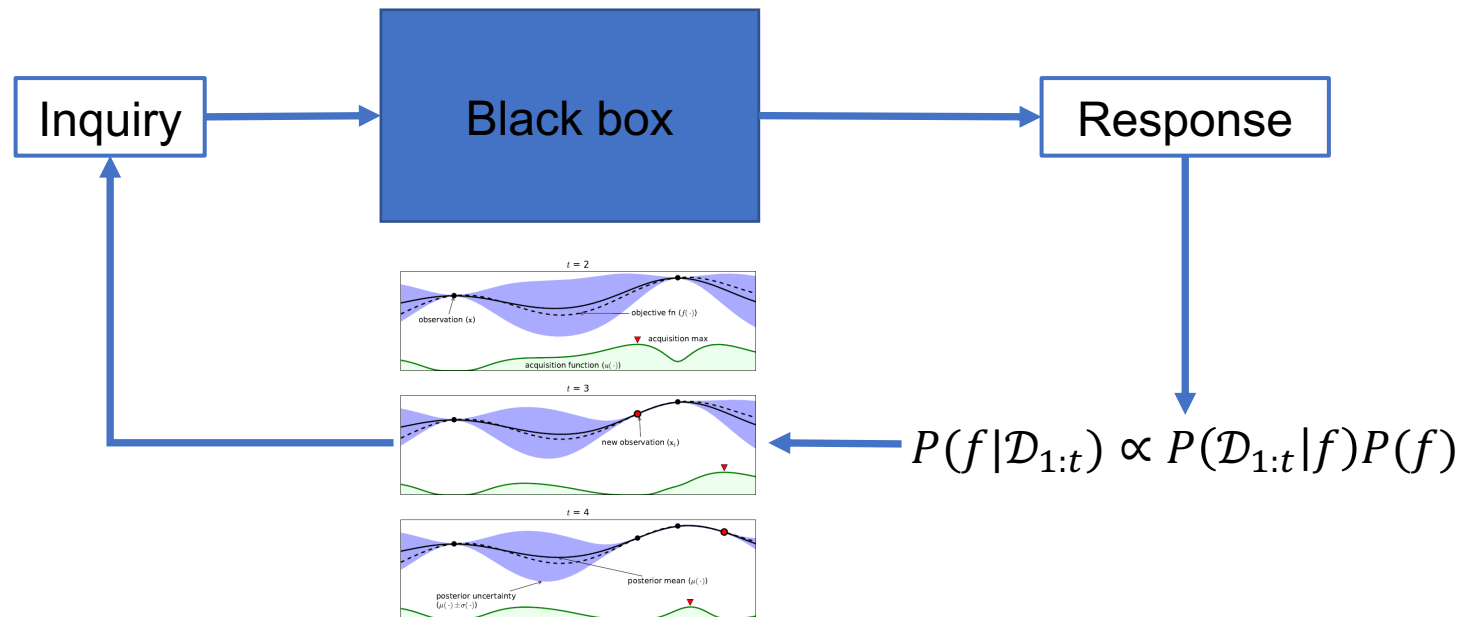
Bayesian Optimization

- Optimizing a nonlinear function $f(\mathbf{x})$ over a compact set $\mathcal{A}$,

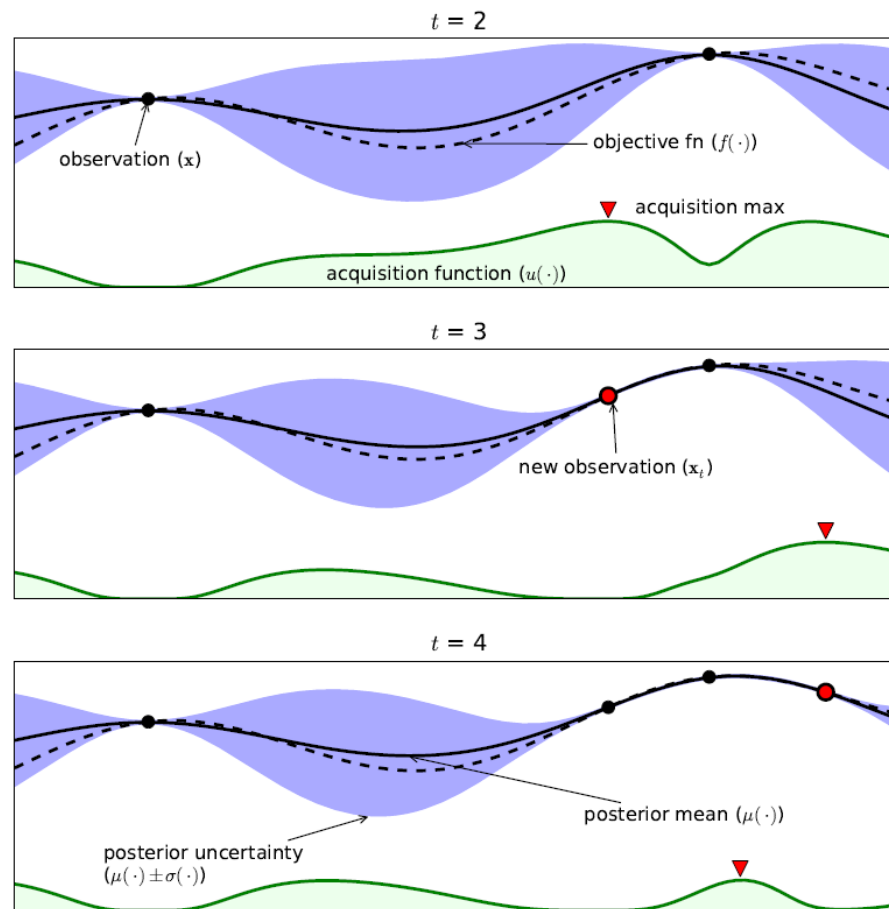
$$\max_{\mathbf{x} \in \mathcal{A} \subset \mathbb{R}^d} f(\mathbf{x})$$

- We assume that the objective function is Lipschitz-continuous. That is, there exists some constant C , such that for all $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{A}$:

$$\|f(\mathbf{x}_1) - f(\mathbf{x}_2)\| \leq C \|\mathbf{x}_1 - \mathbf{x}_2\|$$



Toy Example



Priors over functions

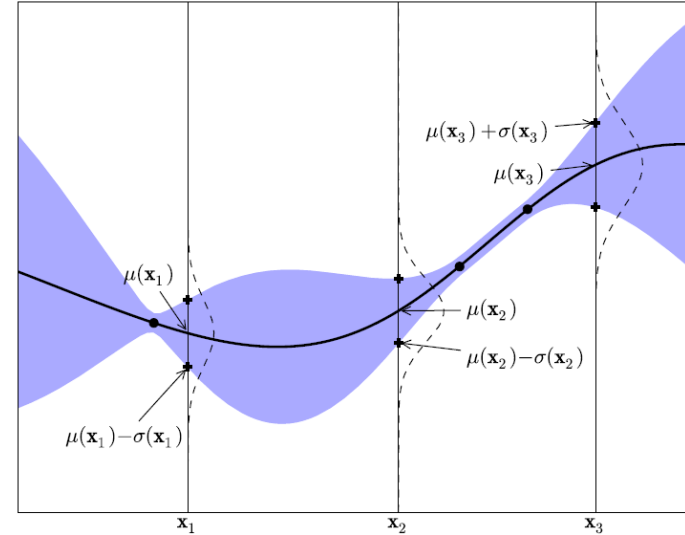
- Gaussian process

$$f(x) \sim \mathcal{GP}((m(x), k(x, x')))$$

- covariance function k

$$k(x_i, x_j) = e^{-\frac{1}{2}\|x_i - x_j\|^2}$$

$$\mathbf{K} = \begin{bmatrix} k(x_1, x_1) & \cdots & k(x_1, x_t) \\ \vdots & \ddots & \vdots \\ k(x_t, x_1) & \cdots & k(x_t, x_t) \end{bmatrix}$$



- Let us denote the value of the function at this arbitrary point as $f_{t+1} = f(x_{t+1})$. Then, by the properties of Gaussian processes, $\mathbf{f}_{1:t}$ and f_{t+1} are jointly Gaussian:

$$\begin{bmatrix} \mathbf{f}_{1:t} \\ f_{t+1} \end{bmatrix} \sim \mathcal{N}(\mathbf{0}, \begin{bmatrix} \mathbf{K} & \mathbf{k} \\ \mathbf{k}^T & k(x_{t+1}, x_{t+1}) \end{bmatrix})$$

- Using the Sherman-Morrison-Woodbury formula

$$P(f_{t+1} | \mathcal{D}_{1:t}, x_{t+1}) = \mathcal{N}(\mu_t(x_{t+1}), \sigma_t^2(x_{t+1}))$$

- where

$$\mu_t(x_{t+1}) = \mathbf{k}^T \mathbf{K}^{-1} \mathbf{f}_{1:t} \quad \sigma_t^2(x_{t+1}) = k(x_{t+1}, x_{t+1}) - \mathbf{k}^T \mathbf{K}^{-1} \mathbf{k}$$

Acquisition Functions



PI

- maximizing the probability of improvement over the incumbent $f(x^+)$,
where $x^+ = \operatorname{argmax}_{x_i \in x_{1:t}} f(x_i)$

$$PI(x) = P(f(x) \geq f(x^+)) = \Phi\left(\frac{\mu(x) - f(x^+)}{\sigma(x)}\right)$$

$$PI(x) = P(f(x) \geq f(x^+) + \xi) = \Phi\left(\frac{\mu(x) - f(x^+) - \xi}{\sigma(x)}\right), \xi \geq 0$$

Acquisition Functions



EI

- defined the improvement function as:

$$I(x) = \max\{0, f_{t+1}(x) - f(x^+)\}$$

- The new query point is found by maximizing the expected improvement

$$x = \operatorname{argmax}_x E(\max\{0, f_{t+1}(x) - f(x^+)\} | \mathcal{D}_t)$$

Acquisition Functions

EI

- The likelihood of improvement I on a normal posterior distribution characterized by $\mu(x), \sigma^2(x)$ can be computed from the normal density function,

$$\frac{1}{\sqrt{2\pi}\sigma(x)} e^{-\frac{(\mu(x)-f(x^+)-I)^2}{2\sigma^2(x)}}$$

- The expected improvement is the integral over this function:

$$\begin{aligned} E(I) &= \int_{I=0}^{I=\infty} I \frac{1}{\sqrt{2\pi}\sigma(x)} e^{-\frac{(\mu(x)-f(x^+)-I)^2}{2\sigma^2(x)}} dI \\ &= \sigma(x) \left[\frac{\mu(x)-f(x^+)}{\sigma(x)} \Phi\left(\frac{\mu(x)-f(x^+)}{\sigma(x)}\right) + \phi\left(\frac{\mu(x)-f(x^+)}{\sigma(x)}\right) \right] \end{aligned}$$

Acquisition Functions

EI

- The expected improvement can be evaluated analytically, yielding:

$$E(I) = \begin{cases} (\mu(x) - f(x^+))\Phi(Z) + \sigma(x)\phi(Z), & \text{if } \sigma(x) > 0 \\ 0, & \text{if } \sigma(x) = 0 \end{cases}$$

$$Z = \frac{\mu(x) - f(x^+)}{\sigma(x)}$$

$$E(I) = \begin{cases} (\mu(x) - f(x^+) - \xi)\Phi(Z) + \sigma(x)\phi(Z), & \text{if } \sigma(x) > 0 \\ 0, & \text{if } \sigma(x) = 0 \end{cases}$$

$$Z = \begin{cases} \frac{\mu(x) - f(x^+) - \xi}{\sigma(x)}, & \text{if } \sigma(x) > 0 \\ 0, & \text{if } \sigma(x) = 0 \end{cases}$$

Acquisition Functions



- Confidence bound criteria

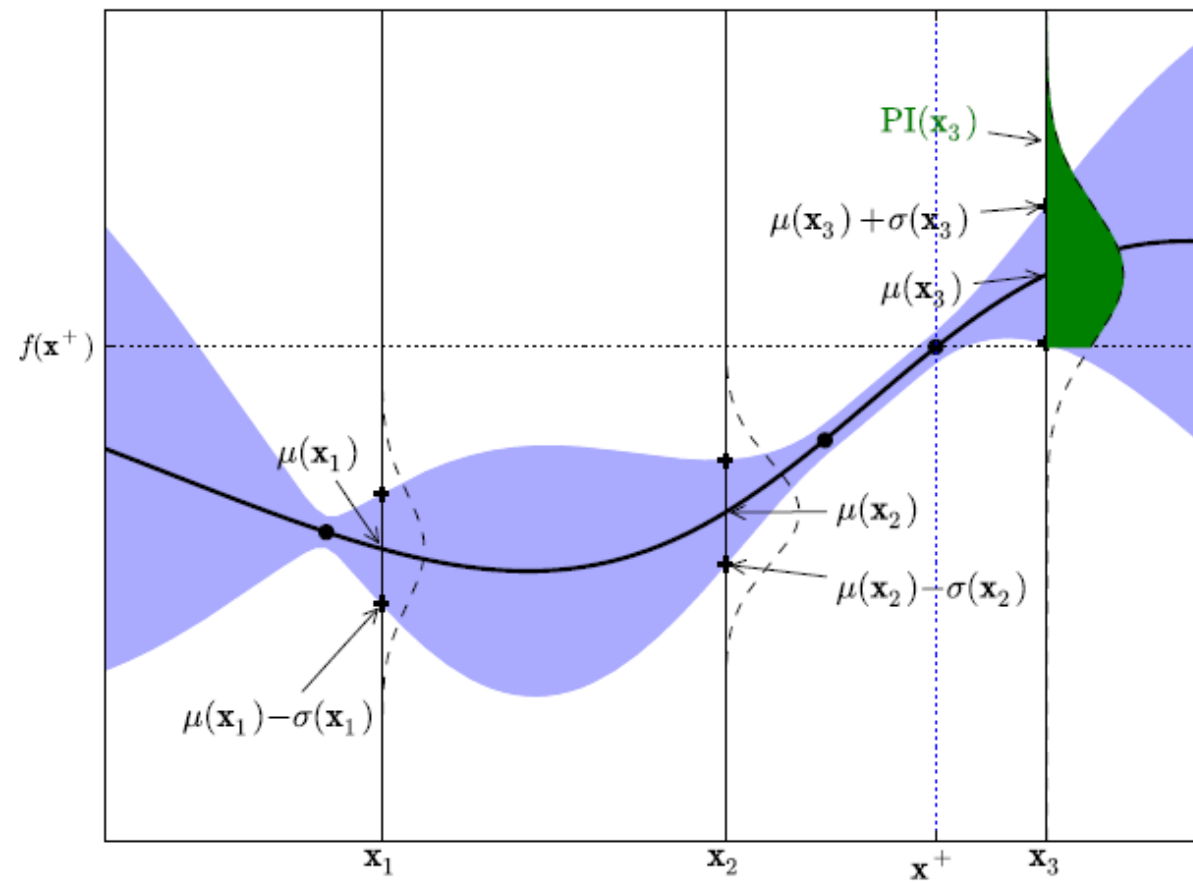
lower confidence bound:

$$LCB(x) = \mu(x) - \kappa\sigma(x), \kappa \geq 0$$

upper confidence bound:

$$UCB(x) = \mu(x) + \kappa\sigma(x), \kappa \geq 0$$

Acquisition Functions



Thanks!
